CI/CD Pipeline with a GitHub Self-Hosted Runner

Markus Ertl · SWE3IL · 13.11.2026

Repository: https://github.com/MarkusErtl7/SWE3IL_CICD

1. Goal and Setup

The goal was to recreate the CI/CD pipeline from the lecture with GitHub Actions. All jobs run on a **self-hosted runner on my own Windows 11 PC** (runner name `ADMIN`) instead of on GitHub-hosted machines. The project is a small Python/Flask web service with pytest unit tests, flake8 linting and a Dockerfile. The pipeline is defined in `.github/workflows/ci-cd.yml` and runs on every push or pull request to `main`.

Stage	Job	Steps
CI	Build & Test	checkout → create venv & install dependencies → flake8 → pytest
CD	Docker Build & Deploy	build image → remove old container → start new container on port 5000 → smoke test on `/health`

The deploy job depends on the CI job (`needs: build-and-test`), so it only runs if all tests pass. Both jobs use `runs-on: self-hosted`.

2. Steps

1. Created a GitHub account and the public repository `SWE3IL_CICD`.
2. Wrote the Flask app, tests, Dockerfile and workflow file and pushed them to `main`.
3. Registered the runner via *Settings* → *Actions* → *Runners* → *New self-hosted runner* (Windows x64): downloaded the runner package, ran `config.cmd` with the registration token and started it with `.\run.cmd`.
4. Started Docker Desktop and checked that Python is available on the PATH.
5. Pushed to `main`: GitHub hands the jobs to my runner, which executes them locally.

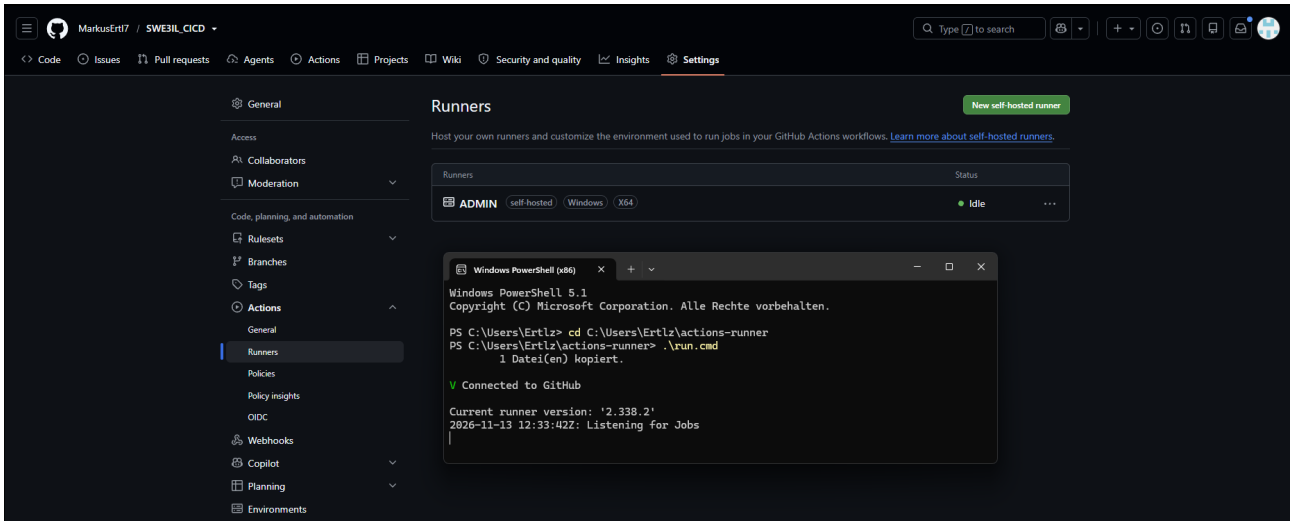


Figure 1: Runner ADMIN registered in GitHub (Idle) and started locally with run.cmd ("Connected to GitHub", "Listening for Jobs").

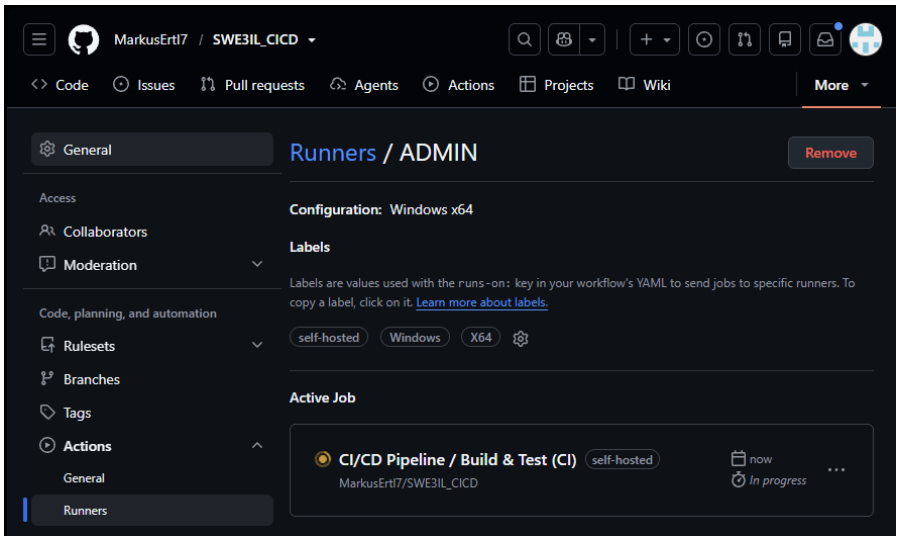


Figure 2: Runner ADMIN (Windows x64) executing the CI job – "Active Job, In progress".

3. Testing the Failure Case

To check that the pipeline really stops broken code, I changed a unit test on purpose (`assert add(2, 3) == 6` instead of `== 5`) and pushed it. The pytest step failed, the CI job turned red and the deploy job was **skipped**, so the running container stayed unchanged. GitHub also sent an e-mail notification.

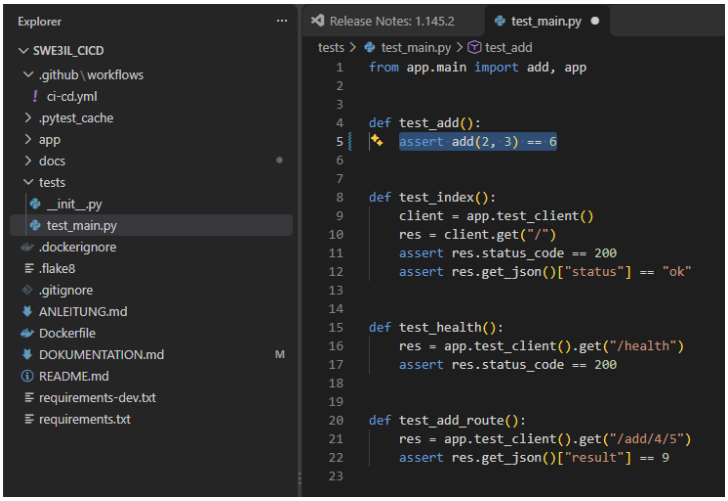


Figure 3: Intentionally broken unit test in tests/test_main.py.

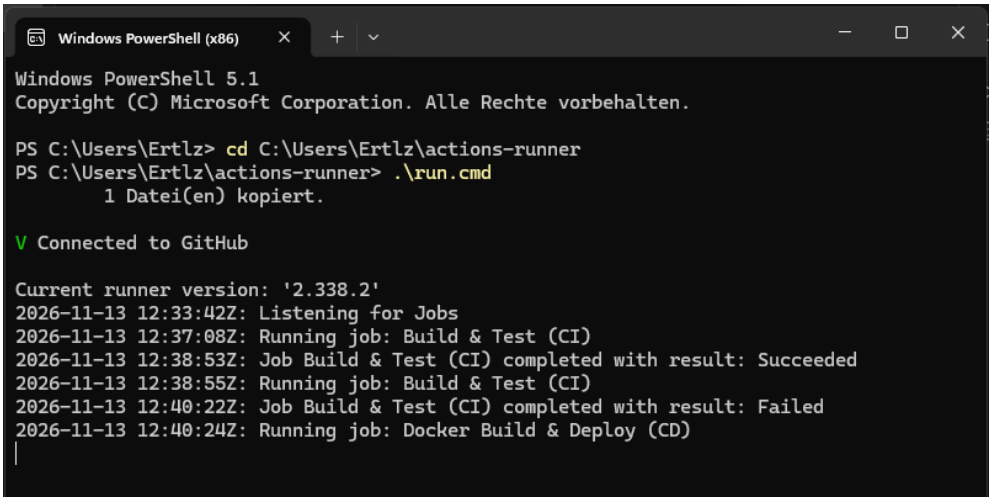


Figure 4: Local runner output – CI succeeded for the previous commit and failed for the broken one. The deploy job at the end belongs to the earlier, correct run.

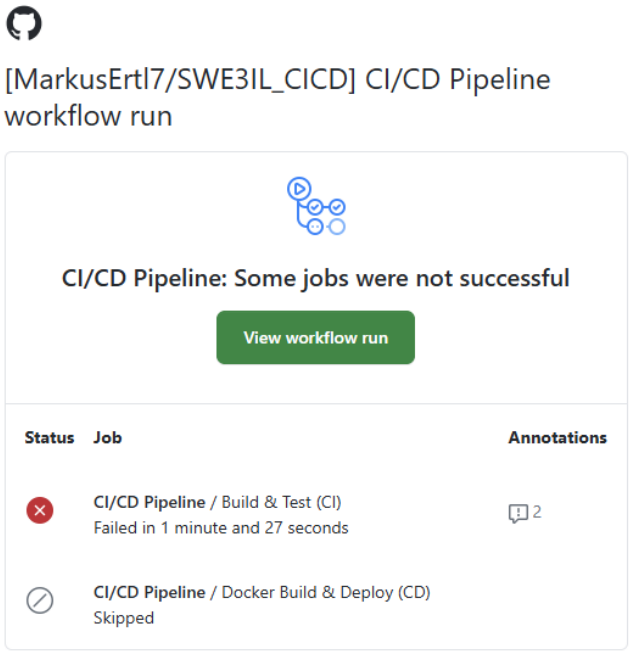


Figure 5: E-mail from GitHub – Build & Test failed, Docker Build & Deploy skipped.

Afterwards the test was reverted and pushed again (commit *good Test*). Run #5 passed the whole pipeline, so the latest run on `main` is successful.

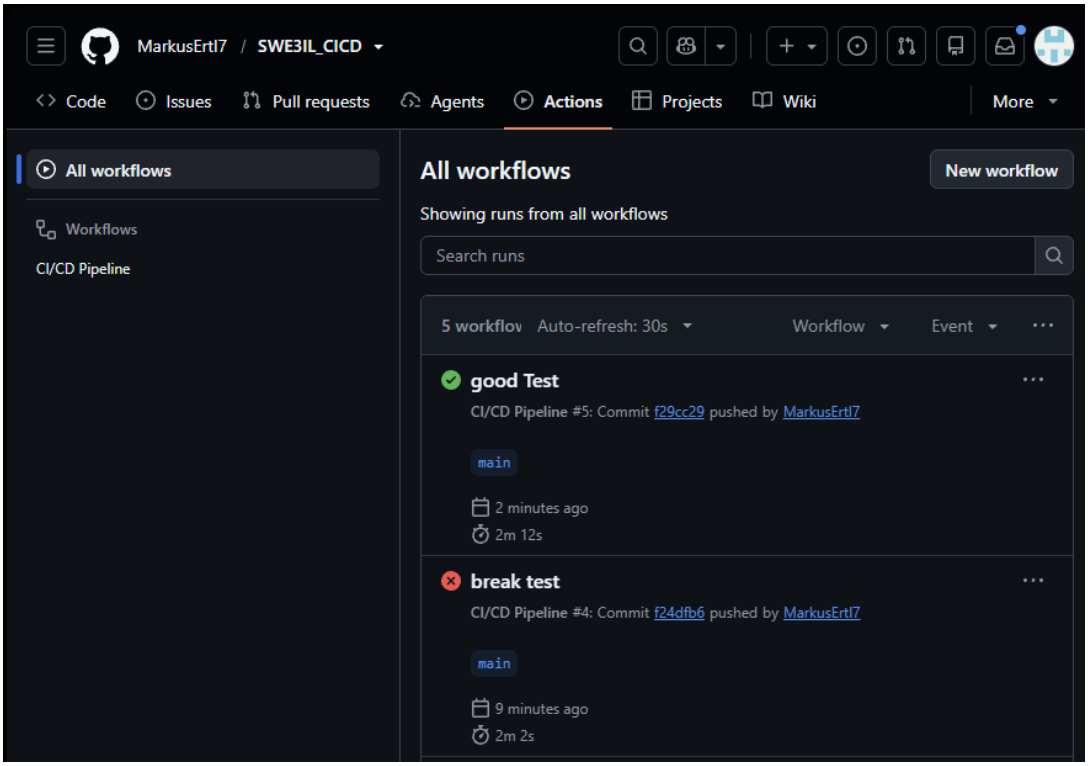


Figure 6: Actions overview – #4 "break test" failed, #5 "good Test" succeeded.

4. Challenges and Solutions

- **First run failed at the first PowerShell step** ("Process completed with exit code 1"). The runner saves every `run:` step as a temporary .ps1 script, and the Windows default ExecutionPolicy (*Restricted*) blocks scripts. Fix: a custom shell in the workflow, `powershell -NoProfile -ExecutionPolicy Bypass -Command ". '{0}'"`, which only applies to the pipeline process and leaves the system setting unchanged.
- **Windows instead of Linux:** the commands from typical examples are bash; on my runner the steps had to be written for PowerShell (e.g. `.\.venv\Scripts\pytest.exe`).
- **Docker access:** the runner is started with `run.cmd` under my user instead of as a Windows service, because the service account has no access to Docker Desktop. Docker Desktop must be running, otherwise the deploy job fails.
- **One job at a time:** a single self-hosted runner processes jobs sequentially. With two quick pushes the deploy job of the first run waited until the CI job of the second run had finished, which looked confusing in the runner output at first.
- **Security:** on a self-hosted runner, workflow code runs directly on my PC. For that reason the deploy job is skipped for pull requests; for real projects a self-hosted runner should only be used with trusted code.

5. Result

The pipeline runs completely on my local machine: every push to `main` is linted and tested, and only if all tests pass, a new Docker image is built and deployed locally on port 5000. A broken test reliably blocks the deployment. The repository contains successful CI/CD runs (e.g. run #5).